

Mateus Leite

Sistema de Login Seguro com Spring Boot, Thymeleaf e MongoDB Atlas

Mogi das Cruzes

2026

Mateus Leite

Sistema de Login Seguro com Spring Boot, Thymeleaf e MongoDB Atlas

Documentação técnica apresentada como atividade individual da disciplina Aplicativos Web, do curso de Sistemas de Informação da Universidade de Mogi das Cruzes.

Universidade de Mogi das Cruzes – UMC

Sistemas de Informação

Orientador: Prof. Alessandro Horas

Mogi das Cruzes

2026

RESUMO

Este documento descreve o desenvolvimento de um sistema web de autenticação e autorização construído com Java 21, Spring Boot 4, Spring Security, Thymeleaf e MongoDB Atlas. O sistema oferece cadastro com validação de dados, login e logout, senhas armazenadas com o algoritmo BCrypt, proteção contra CSRF e controle de acesso baseado em três perfis: aluno, orientador e administrador. Os usuários e as sessões HTTP são persistidos em um cluster gerenciado do MongoDB Atlas, o que permite manter o usuário autenticado mesmo após a reinicialização da aplicação. A interface foi construída com Thymeleaf e separa o design da lógica de negócio por meio de um layout base e de temas configuráveis baseados em variáveis CSS. O projeto foi organizado por funcionalidade e versionado segundo o modelo Gitflow, e foi pensado para que o seu modelo de perfis, usuários e identidade visual possa ser reaproveitado no Projeto de Final de Curso Athena, que utiliza AWS Cognito.

Palavras-chave: autenticação; Spring Security; MongoDB Atlas; controle de acesso; segurança da informação.

SUMÁRIO

1	Introdução	4
2	Estrutura do sistema	5
2.1	Tecnologias utilizadas	5
2.2	Organização do código	5
2.3	Perfis e controle de acesso	6
2.4	Modelo de dados	7
2.5	Cadastro e validação	7
3	Integração com o MongoDB Atlas	8
3.1	Configuração do cluster	8
3.2	Conexão segura e segredos	8
3.3	Coleções e índices	8
4	Decisões de design	10
4.1	Segurança	10
4.2	Interface e temas	11
4.3	Adaptação ao PFC e ao AWS Cognito	11
4.4	Testes e versionamento	11
5	Conclusão	13
	Referências	14

1 INTRODUÇÃO

A autenticação (verificar quem é o usuário) e a autorização (definir o que ele pode acessar) estão presentes em praticamente todo sistema web. Por serem a porta de entrada da aplicação, falhas nesses mecanismos expõem dados pessoais e funções administrativas, o que torna essencial adotar práticas consolidadas, como o armazenamento de senhas com funções de hash adaptativas e a proteção contra requisições forjadas (OWASP FOUNDATION, 2026a; 2026b).

Este trabalho apresenta um sistema de login seguro desenvolvido com o ecossistema Spring e o banco de dados MongoDB, hospedado no serviço gerenciado MongoDB Atlas. O objetivo foi construir “o básico muito bem feito”: cadastro, login e logout com hash de senha e validação de dados; controle de acesso por perfis; persistência de usuários e sessões na nuvem; e uma interface agradável, acessível e preparada para temas visuais personalizados.

Além de cumprir os requisitos da atividade, o projeto foi planejado como base de estudo para o Projeto de Final de Curso (PFC) *Athena – O Portal do PFC*. Como o PFC usa outra pilha tecnológica (Go, AWS Lambda, Angular, DynamoDB e AWS Cognito), não se trata de reaproveitar o código diretamente, mas sim o modelo: os perfis de acesso, os campos do usuário, as regras de cadastro e a identidade visual.

As próximas seções descrevem a estrutura do sistema, a integração com o MongoDB Atlas e as principais decisões de design, com ênfase em segurança, separação entre interface e lógica e adaptação futura ao PFC.

2 ESTRUTURA DO SISTEMA

2.1 TECNOLOGIAS UTILIZADAS

O sistema foi desenvolvido em Java 21 (versão de suporte estendido) com Spring Boot 4.0.8, que reúne o Spring MVC (camada web), o Spring Security 7 (autenticação e autorização) e o Spring Data MongoDB (acesso ao banco). As páginas são geradas no servidor com Thymeleaf, e o Thymeleaf Layout Dialect permite reutilizar um layout base (SPRING, 2026a; THYMELEAF, 2026). O build usa o Maven Wrapper, que dispensa a instalação do Maven.

A escolha da versão do Spring Boot foi guiada pela compatibilidade com o armazenamento de sessões no MongoDB. O módulo antigo, `spring-session-data-mongodb`, deixou de ser mantido pelo time do Spring na versão 3.5 e passou à MongoDB Inc. como `org.mongodb:mongodb-spring-session`. A versão 4.0.0 desse artefato foi construída sobre Spring Session 4.0, Spring Security 7.0 e Spring Data 2025.1, exatamente as versões que acompanham o Spring Boot 4.0.x (MONGODB, 2025). Escolher essa combinação evita conflitos de versões entre bibliotecas.

2.2 ORGANIZAÇÃO DO CÓDIGO

O código foi organizado por funcionalidade, e não por tipo de classe, para que cada pacote agrupe tudo o que diz respeito a um assunto:

- **config**: configuração de segurança (`SecurityConfig`), de sessões (`SessionConfig`), propriedades da aplicação (`AppProperties`), criação do administrador inicial (`AdminSeeder`) e tema visual (`Tema`, `ThemeAdvice`);
- **user**: o documento `User`, os enumerados `Role` e `Status`, o repositório e o `UserService`, que concentra as regras de negócio;
- **auth**: login e cadastro (`AuthController`), formulário e validações do cadastro e a integração com o Spring Security (`MongoUserDetailsService`, `UsuarioLogado`);
- **web**: os controladores das páginas de cada perfil.

Os templates ficam separados em layout (`layouts/base.html`), fragmentos reutilizáveis (cabeçalho, rodapé e alertas), páginas e páginas de erro. Os arquivos estáticos ficam divididos entre a estrutura visual (`css/base.css`) e os temas (`themes/<nome>/theme.css`).

2.3 PERFIS E CONTROLE DE ACESSO

O sistema possui três perfis, definidos no enumerado `Role`:

- **ALUNO**: criado pelo cadastro público, que sempre atribui esse perfil, independentemente do que for enviado na requisição;
- **ORIENTADOR**: criado apenas pelo administrador, no painel administrativo;
- **ADMIN**: o primeiro é criado automaticamente na inicialização, com e-mail e senha lidos de variáveis de ambiente; os demais podem ser criados por outro administrador.

Cada perfil tem a sua área (`/aluno/**`, `/orientador/**` e `/admin/**`), e o acesso é verificado pelo Spring Security a partir da role do usuário. A rota `/dashboard` redireciona cada usuário para a área do seu perfil. Quem tenta acessar a área de outro perfil recebe uma página amigável de acesso negado (código HTTP 403), e visitantes não autenticados são redirecionados ao login. A Figura 1 mostra o painel do administrador, que lista os usuários e permite criar orientadores e administradores e ativar ou desativar contas.

Figura 1 – Painel do administrador (tema Athena)

NOME	PERFIL	SITUAÇÃO
Administrador admin.teste@exemplo.com	Administrador	■ Ativa Você
Aluno Teste aluno.teste@exemplo.com	Aluno	■ Ativa Desativar
Orientador Teste orientador.teste@exemplo.com	Orientador	■ Ativa Desativar

Fonte: elaborado pelo autor (2026).

2.4 MODELO DE DADOS

Os usuários são armazenados na coleção **users**. Cada documento contém: identificador, nome, e-mail (usado como login), hash da senha, perfil, situação (**ATIVO** ou **INATIVO**), indicador de e-mail confirmado e data de criação. Seguindo o princípio da minimização de dados da Lei Geral de Proteção de Dados Pessoais (BRASIL, 2018), não são coletados dados desnecessários, como CPF ou telefone. O campo de e-mail confirmado já existe no modelo, mas a confirmação por e-mail foi deixada como trabalho futuro.

2.5 CADASTRO E VALIDAÇÃO

O formulário de cadastro valida, no servidor: nome obrigatório; e-mail em formato válido e ainda não cadastrado; senha com no mínimo 8 caracteres, contendo letras e números; e confirmação de senha igual à senha. As regras de formato são declaradas com anotações do Bean Validation, incluindo a anotação própria **@SenhaForte**, e as regras que dependem do banco ou da configuração (e-mail duplicado e domínio permitido) ficam no **RegisterValidator**.

Os erros aparecem ao lado de cada campo, em português, e o formulário mantém os dados digitados, exceto as senhas, que nunca são devolvidas à tela (Figura 2). Opcionalmente, a propriedade **app.registration.allowed-domain** restringe o cadastro a um domínio institucional, como **umc.br**, aceitando também os seus subdomínios.

Figura 2 – Mensagens de validação no cadastro

Athena
O Portal do PFC

Entrar Criar conta

Criar conta

Preencha seus dados para acessar a área do aluno.

Nome completo

E-mail

Informe um e-mail válido, como nome@exemplo.com.

Senha

Mínimo de 8 caracteres, com letras e números.
Use de 8 a 72 caracteres, com pelo menos uma letra e um número.

Confirme a senha

Repita a senha.

Criar conta

Já tem conta? [Entrar](#)

© 2026 Mateus Leite - Projeto acadêmico com Spring Boot, Thymeleaf e MongoDB Atlas

Fonte: elaborado pelo autor (2026).

3 INTEGRAÇÃO COM O MONGODB ATLAS

O MongoDB Atlas é o serviço de banco de dados em nuvem da MongoDB Inc. Neste projeto foi usado um cluster gratuito (camada M0) hospedado na AWS, região de São Paulo (`sa-east-1`), escolhida pela menor latência (MONGODB, 2026).

3.1 CONFIGURAÇÃO DO CLUSTER

A configuração seguiu quatro passos: (i) criação do cluster M0; (ii) criação de um usuário de banco com a permissão *Read and write to any database*, aplicando o princípio do menor privilégio, já que a aplicação não precisa administrar o cluster; (iii) liberação, na lista de acesso por IP (*IP Access List*), apenas dos endereços de saída da rede do desenvolvedor; e (iv) obtenção da *connection string* no formato `mongodb+srv://`, que habilita criptografia TLS por padrão.

Durante o desenvolvimento, a rede utilizada alternava entre dois endereços IP públicos. Como apenas um deles estava liberado, parte das conexões era recusada pelo Atlas com o erro `SSLException: Received fatal alert: internal_error`, de forma intermitente. A solução foi liberar ambos os endereços, mantendo a restrição por IP em vez de liberar o acesso para qualquer origem (`0.0.0.0/0`).

3.2 CONEXÃO SEGURA E SEGREDOS

A *connection string* contém credenciais e, por isso, nunca é versionada. O arquivo `application.yml` apenas referencia a variável `${MONGODB_URI}`, e o valor real fica em um arquivo `.env` local, carregado automaticamente pela propriedade `spring.config.import` e ignorado pelo Git. O repositório contém apenas o modelo `.env.example`, com valores fictícios. O mesmo mecanismo fornece o e-mail e a senha do primeiro administrador.

3.3 COLEÇÕES E ÍNDICES

A aplicação cria e utiliza duas coleções no banco `login_seguro`:

- **users**: armazena os usuários. Um índice único no campo `email`, declarado com `@Indexed(unique = true)` e criado na inicialização, garante no próprio banco que não existam dois usuários com o mesmo e-mail, mesmo em cadastros simultâneos;

- **sessions**: armazena as sessões HTTP, por meio do Spring Session com o módulo **mongodb-spring-session**. Um índice TTL (*time to live*) no campo **expireAt** faz o próprio MongoDB remover as sessões expiradas, configuradas para 30 minutos de inatividade.

Guardar as sessões no banco, e não na memória do servidor, faz com que o usuário continue autenticado mesmo após a reinicialização da aplicação, e permite que várias instâncias compartilhem as mesmas sessões. Esse comportamento foi verificado durante os testes: após reiniciar a aplicação, a sessão continuou válida; após o logout, o documento correspondente foi removido da coleção.

4 DECISÕES DE DESIGN

4.1 SEGURANÇA

As principais decisões de segurança foram:

- a) **Hash de senhas com BCrypt.** As senhas nunca são armazenadas em texto puro. O BCrypt é uma função de hash adaptativa, propositalmente lenta e com sal aleatório, o que dificulta ataques de dicionário e de força bruta (PROVOS; MAZIÈRES, 1999; OWASP FOUNDATION, 2026a). Como o algoritmo considera apenas os primeiros 72 bytes, a validação limita o tamanho da senha;
- b) **Proteção contra CSRF.** Todo formulário enviado por POST carrega um token secreto, inserido automaticamente pelo Thymeleaf, e requisições sem o token são recusadas (OWASP FOUNDATION, 2026b). Pelo mesmo motivo, o logout é feito via POST;
- c) **Perfil definido pelo servidor.** O formulário de cadastro não possui campo de perfil, e o serviço sempre cria um ALUNO. Assim, adulterar a requisição não permite obter privilégios maiores;
- d) **Mensagens que não vazam informação.** E-mail inexistente e senha incorreta produzem a mesma mensagem. Além disso, a verificação de conta inativa foi configurada para ocorrer somente após a senha ser conferida, de modo que quem não conhece a senha não descobre a situação da conta;
- e) **Contas inativas e proteção do administrador.** Usuários inativos não conseguem entrar, e um administrador não pode desativar a própria conta, o que evita deixar o sistema sem administração;
- f) **Gestão de segredos e menor privilégio.** Credenciais ficam apenas em variáveis de ambiente, e o usuário do banco tem somente as permissões necessárias.

Essas decisões se apoiam nos mecanismos padrão do Spring Security (SPRING, 2026b). A implementação de `UserDetailsService` busca os usuários no MongoDB, e o objeto do usuário autenticado (`UsuarioLogado`) descarta o hash da senha após o login, para que ele não fique guardado na sessão.

4.2 INTERFACE E TEMAS

A interface foi construída com Thymeleaf e um layout base (`layouts/base.html`) que reúne cabeçalho, rodapé e mensagens de alerta. Cada página declara apenas o seu conteúdo. O cabeçalho exibe o nome e o perfil do usuário autenticado e o botão de sair.

O design foi separado da lógica em três camadas: os templates usam apenas classes semânticas (por exemplo, `.card`, `.btn-primary` e `.alert-error`); o arquivo `base.css` define a estrutura e os componentes sem nenhuma cor fixa; e cada tema define apenas variáveis CSS (cores, fontes e arredondamento). O tema ativo é escolhido pela propriedade `app.theme`, então trocar de tema exige mudar uma única linha de configuração. Foram criados dois temas (Figura 3):

- **default**: neutro e moderno, com as fontes do sistema operacional;
- **athena**: identidade visual do PFC, com azul-marinho (`#23398A`), bronze (`#A97A46`), fundo creme, fontes Playfair Display e Inter e cantos retos.

As telas são responsivas e seguem boas práticas de acessibilidade: rótulos associados a todos os campos, foco visível na navegação por teclado, link para pular direto ao conteúdo, mensagens de erro ligadas aos campos e contraste adequado entre texto e fundo (W3C, 2023). Não foram usados frameworks CSS, o que mantém o projeto leve e o código fácil de entender.

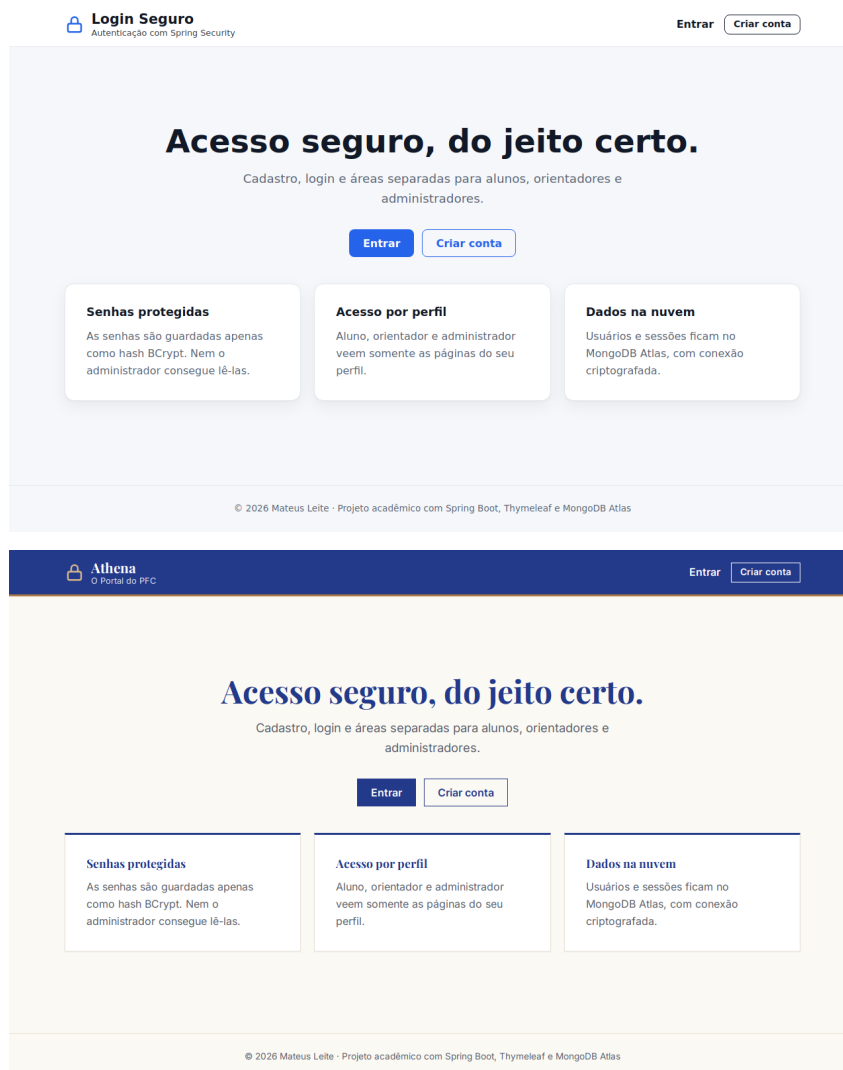
4.3 ADAPTAÇÃO AO PFC E AO AWS COGNITO

As regras de acesso foram escritas de forma a depender somente do perfil do usuário, e não da forma de autenticação. No PFC, a autenticação é feita pelo AWS Cognito, com tokens JWT e o perfil informado no atributo `custom:perfil` (AMAZON WEB SERVICES, 2026). Para adaptar este sistema a esse cenário, basta alterar a classe `SecurityConfig`: substituir o login por formulário por login OIDC ou validação de JWT e converter o atributo `custom:perfil` nas authorities `ROLE_ALUNO`, `ROLE_ORIENTADOR` e `ROLE_ADMIN`. As regras de autorização, os controladores e as páginas permanecem inalterados. O redirecionamento do `/dashboard`, por exemplo, consulta apenas as authorities do usuário.

4.4 TESTES E VERSIONAMENTO

Foram escritos testes automatizados para os comportamentos essenciais: o cadastro público sempre cria um aluno, mesmo quando a requisição tenta enviar outro perfil; a senha é salva como hash BCrypt; um aluno não acessa a área administrativa; um usuário anônimo é redirecionado ao login; formulários sem token CSRF são recusados; e o administrador

Figura 3 – Página inicial nos temas default (acima) e athena (abaixo)



Fonte: elaborado pelo autor (2026).

não pode desativar a si mesmo. Os testes simulam o banco de dados, então executam rapidamente e sem depender do Atlas.

O código foi versionado com Git seguindo o modelo Gitflow (DRIESSEN, 2010): a branch **main** contém as versões publicadas, a **develop** integra o trabalho, e cada etapa foi desenvolvida em uma branch **feature/***, integrada com *merge* sem *fast-forward*. As mensagens de commit seguem o padrão *Conventional Commits*.

5 CONCLUSÃO

O sistema desenvolvido atende aos requisitos propostos: cadastro, login e logout com senhas protegidas por BCrypt e validação de dados; controle de acesso com três perfis; uso do Spring Security para autenticação e autorização; armazenamento de usuários e sessões no MongoDB Atlas, com conexão criptografada e segredos fora do repositório; e uma interface em Thymeleaf com o design separado da lógica e temas configuráveis.

Entre os aprendizados, destacam-se a importância de verificar a compatibilidade entre versões de bibliotecas, evidenciada pela mudança de mantenedor do módulo de sessões no MongoDB, e o diagnóstico de problemas de rede na integração com serviços em nuvem, como a lista de acesso por IP do Atlas.

Como trabalhos futuros, propõem-se: confirmação de e-mail, recuperação de senha, encerramento imediato das sessões de um usuário desativado, limite de tentativas de login e a integração com o AWS Cognito, aproximando o projeto da arquitetura do PFC Athena.

REFERÊNCIAS

- AMAZON WEB SERVICES. **Amazon Cognito Developer Guide**. Seattle: AWS, 2026. Disponível em: <https://docs.aws.amazon.com/cognito/>. Acesso em: 8 out. 2026.
- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.
- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.
- BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 8 out. 2026.
- DRIESSEN, Vincent. **A successful Git branching model**. [S. l.], 2010. Disponível em: <https://nvie.com/posts/a-successful-git-branching-model/>. Acesso em: 8 out. 2026.
- MONGODB. **MongoDB Atlas Documentation**. New York: MongoDB, 2026. Disponível em: <https://www.mongodb.com/docs/atlas/>. Acesso em: 8 out. 2026.
- MONGODB. **mongodb-spring-session 4.0.0**. [S. l.]: Maven Central Repository, 2025. Disponível em: <https://repo1.maven.org/maven2/org/mongodb/mongodb-spring-session/>. Acesso em: 8 out. 2026.
- OWASP FOUNDATION. **Password Storage Cheat Sheet**. [S. l.], 2026a. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 8 out. 2026.
- OWASP FOUNDATION. **Cross-Site Request Forgery Prevention Cheat Sheet**. [S. l.], 2026b. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html. Acesso em: 8 out. 2026.
- PROVOS, Niels; MAZIÈRES, David. A future-adaptable password scheme. *In*: USENIX ANNUAL TECHNICAL CONFERENCE, 1999, Monterey. **Proceedings** [...]. Berkeley: USENIX Association, 1999. p. 81-91.
- SPRING. **Spring Boot Reference Documentation**. [S. l.]: Broadcom, 2026a. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 8 out. 2026.
- SPRING. **Spring Security Reference**. [S. l.]: Broadcom, 2026b. Disponível em: <https://docs.spring.io/spring-security/reference/>. Acesso em: 8 out. 2026.

THYMELEAF. **Tutorial: Using Thymeleaf.** [S. l.], 2026. Disponível em: <https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>. Acesso em: 8 out. 2026.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.2.** [S. l.]: World Wide Web Consortium, 2023. Disponível em: <https://www.w3.org/TR/WCAG22/>. Acesso em: 8 out. 2026.